

1.5 Task 4 - Graph Neural Network (GNN)

In this task we model API-call sequences as graphs and train GNN architectures (GCN, GraphSAGE, GAT) to classify samples. We reuse the lab preprocessing and reporting utilities for consistency.

```
[ ]: # Create directory for plots
save_dir = results_path + 'images/' + 'task4_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.5.1 Modeling API Sequences as Graphs

```
[ ]: df_train
```

```
[ ]:
      api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2      [FindResourceExW, LoadResource, FindResourceEx...      1
3      [FindResourceExW, LoadResource, FindResourceEx...      1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH...      1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N...      1

      seq_length      seq_fixed  \
0              73  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...
1              88  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
2              79  [FindResourceExW, LoadResource, FindResourceEx...
3              71  [FindResourceExW, LoadResource, FindResourceEx...
4              63  [LdrGetProcedureAddress, SetErrorMode, LdrLoad...
...
16320          64  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...
16321          78  [NtClose, LdrGetProcedureAddress, CryptCreateH...
16322          84  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...
16323          62  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...
16324          77  [SetFilePointer, NtReadFile, SetFilePointer, N...

      seq_sequential_ids
0      [117, 118, 119, 118, 117, 118, 86, 123, 201, 2...
1      [133, 119, 118, 82, 80, 137, 82, 152, 153, 163...
2      [46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
3      [46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
4      [118, 219, 119, 219, 81, 46, 121, 46, 121, 46,...
...
16320  [118, 119, 118, 119, 118, 119, 118, 119, 118, ...
```

```

16321  [134, 118, 24, 118, 30, 118, 135, 66, 221, 166...
16322  [118, 117, 118, 81, 216, 119, 118, 46, 121, 46...
16323  [118, 117, 118, 117, 118, 117, 118, 117, 118, ...
16324  [221, 166, 221, 166, 221, 45, 121, 133, 146, 1...

```

[16325 rows x 5 columns]

```

[ ]: df_train_split, df_val = train_test_split(
    df_train,
    test_size=0.2,
    random_state=42,
    stratify=df_train['is_malware'], # Preserve class balance
    shuffle=True
)

```

```

[ ]: # Use original api_call_sequence (not truncated seq_sequential_ids)
# GNNs can handle variable-length graphs, so we use full sequences
X_train_split = df_train_split['api_call_sequence'].values
y_train_split = df_train_split['is_malware'].values

X_val = df_val['api_call_sequence'].values
y_val = df_val['is_malware'].values

X_test = df_test['api_call_sequence'].values
y_test = df_test['is_malware'].values

print(f"Train length: {len(X_train_split)}")
print(f"Val length: {len(X_val)}")
print(f"Test length: {len(X_test)}")

```

Train length: 13060

Val length: 3265

Test length: 6505

```

[ ]: # --- Graph Construction ---

def create_graph_from_sequence(sequence, api_to_id, label, window_size=1):
    """
    Create a sparse graph containing ONLY the nodes present in the sequence.
    Accepts API name strings and converts them to integer IDs using api_to_id_
    ↪mapping.
    Unknown APIs are mapped to UNK_IDX (index 1).
    """
    UNK_IDX = 1 # Same as defined in Task 3

    # 1. Convert API names to integer IDs (handle unknown APIs with UNK_IDX)
    sequence_ids = [api_to_id.get(api, UNK_IDX) for api in sequence]

```

```

# 2. Identify unique node IDs present in this sequence
# IMPORTANT: Filter out padding (0) before creating node set
unique_api_ids = sorted(list(set([api_id for api_id in sequence_ids if
↪api_id != 0])))
num_nodes = len(unique_api_ids)

# 3. Create a mapping from Global API ID -> Local Graph Node Index
global_to_local = {global_id: local_id for local_id, global_id in
↪enumerate(unique_api_ids)}

# 4. Create Node Features (The Global IDs to be fed into the Embedding
↪Layer)
x = torch.tensor(unique_api_ids, dtype=torch.long).unsqueeze(1)

# 5. Create Edges (Transition Graph)
src_list = []
dst_list = []

# Sliding window to find transitions
for i in range(len(sequence_ids) - window_size):
    global_src = sequence_ids[i]
    global_dst = sequence_ids[i + window_size]

    # Skip padding (0) if present
    if global_src == 0 or global_dst == 0:
        continue

    # Map to local indices
    src_list.append(global_to_local[global_src])
    dst_list.append(global_to_local[global_dst])

edge_index = torch.tensor([src_list, dst_list], dtype=torch.long)

data = Data(
    x=x,
    edge_index=edge_index,
    y=torch.tensor([label], dtype=torch.float32),
    num_nodes=num_nodes
)

return data

def create_graphs_from_sequences(sequences, api_to_id, labels, window_size=1):
    graph_data_list = []
    for sequence, label in zip(sequences, labels):
        # Ensure sequence is a list (handles numpy arrays)
        seq_list = sequence if isinstance(sequence, list) else list(sequence)

```

```

        data = create_graph_from_sequence(seq_list, api_to_id, label,
        ↪window_size)
        graph_data_list.append(data)

        # Sanity check for each graph
        if data.edge_index.numel() > 0 and data.edge_index.max().item() >= data.
        ↪num_nodes:
            print(f"Warning: Graph {idx} has invalid edge index!")

        return graph_data_list

# --- Execute Creation ---
print("Creating sparse graphs...")
train_graphs = create_graphs_from_sequences(X_train_split, api_to_id,
        ↪y_train_split)
val_graphs = create_graphs_from_sequences(X_val, api_to_id, y_val)
test_graphs = create_graphs_from_sequences(X_test, api_to_id, y_test)

print(f"Training graphs: {len(train_graphs)}")
print(f"First graph nodes: {train_graphs[0].num_nodes} (Subset of total vocab)")
print(f"First graph edges: {train_graphs[0].num_edges}")

```

Creating sparse graphs...

Training graphs: 13060

First graph nodes: 21 (Subset of total vocab)

First graph edges: 80

```

[ ]: import networkx as nx
import matplotlib.pyplot as plt
from torch_geometric.utils import to_networkx

def visualize_graph(data, id_to_api_mapping=None):
    # Convert PyG data to NetworkX
    G = to_networkx(data, to_undirected=False) # Keep it directed to see flow

    plt.figure(figsize=(12, 8))

    # Layout (Spring layout tends to show clusters and loops well)
    pos = nx.spring_layout(G, seed=42)

    # Draw nodes
    nx.draw_networkx_nodes(G, pos, node_size=800, node_color='lightblue',
        ↪alpha=0.8)

    # Draw edges (with arrows)
    nx.draw_networkx_edges(G, pos, width=1.0, alpha=0.5, arrowstyle='->',
        ↪arrowsize=20)

```

```

# Labels: If you have the mapping, show API names. Otherwise show IDs.
if id_to_api_mapping:
    # Get the Global API ID from data.x for each local node index
    labels = {i: id_to_api_mapping.get(data.x[i].item(), str(i)) for i in
↪range(data.num_nodes)}
    nx.draw_networkx_labels(G, pos, labels, font_size=10,
↪font_weight="bold")
else:
    nx.draw_networkx_labels(G, pos)

plt.title(f"Malware Execution Graph\nLabel: {data.y.item()} (1=Malware,
↪0=Benign)")
plt.axis('off')
plt.show()

# --- RUN THE CHECK ---
# Plot example graphs from each dataset
print("Plotting example graphs...\n")
print("Training set example:")
visualize_graph(train_graphs[0], id_to_api)

print("\nValidation set example:")
visualize_graph(val_graphs[0], id_to_api)

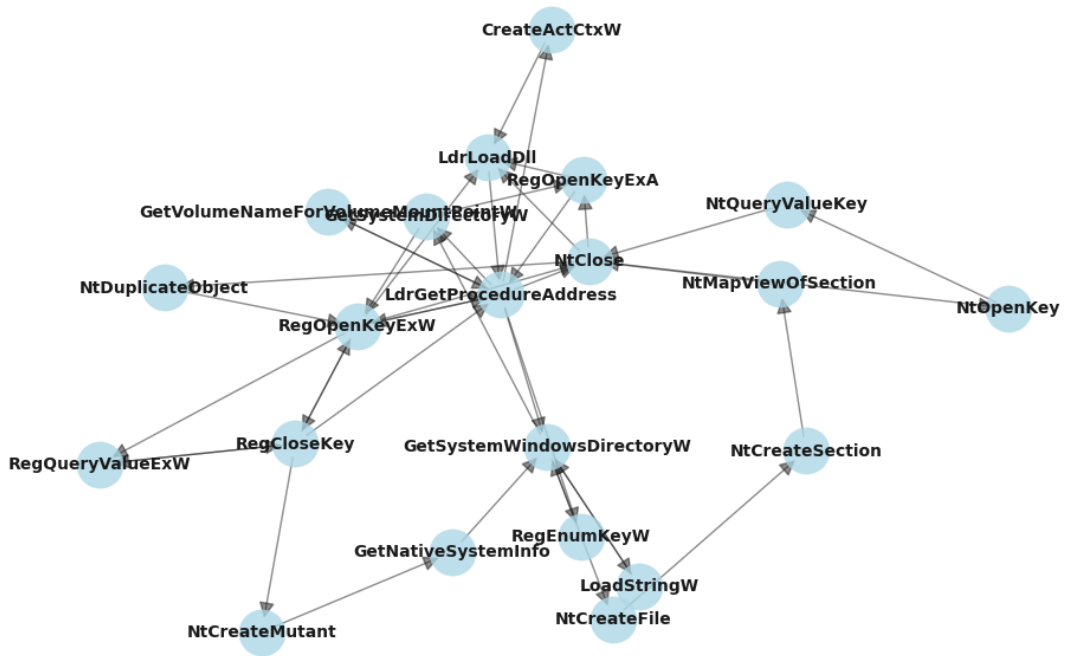
print("\nTest set example:")
visualize_graph(test_graphs[0], id_to_api)

```

Plotting example graphs...

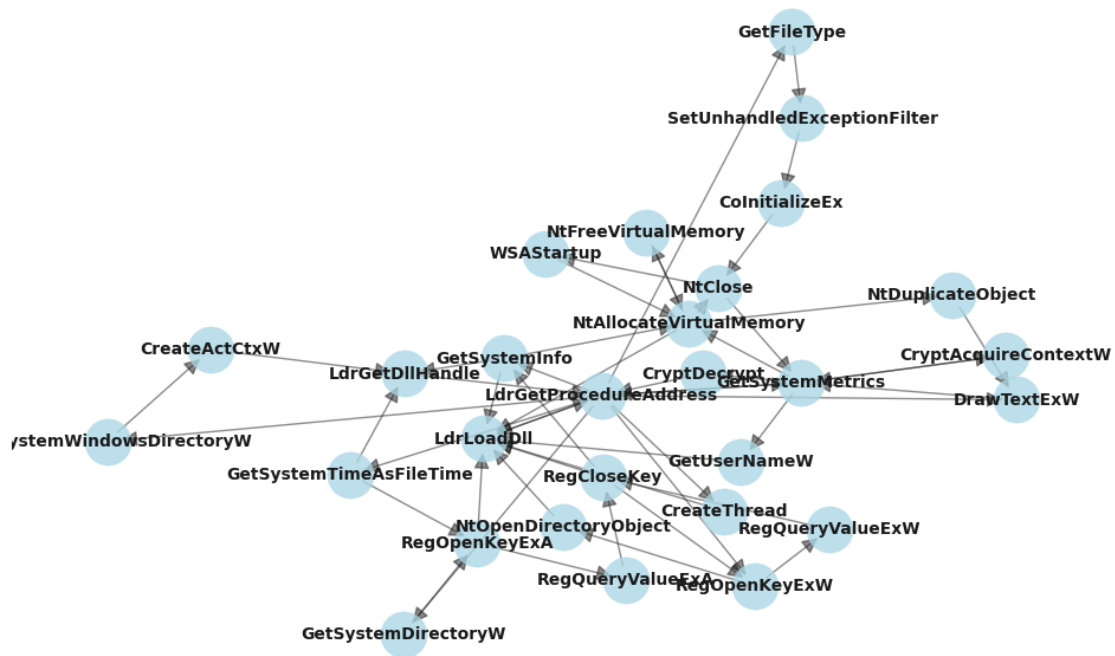
Training set example:

Malware Execution Graph
Label: 1.0 (1=Malware, 0=Benign)



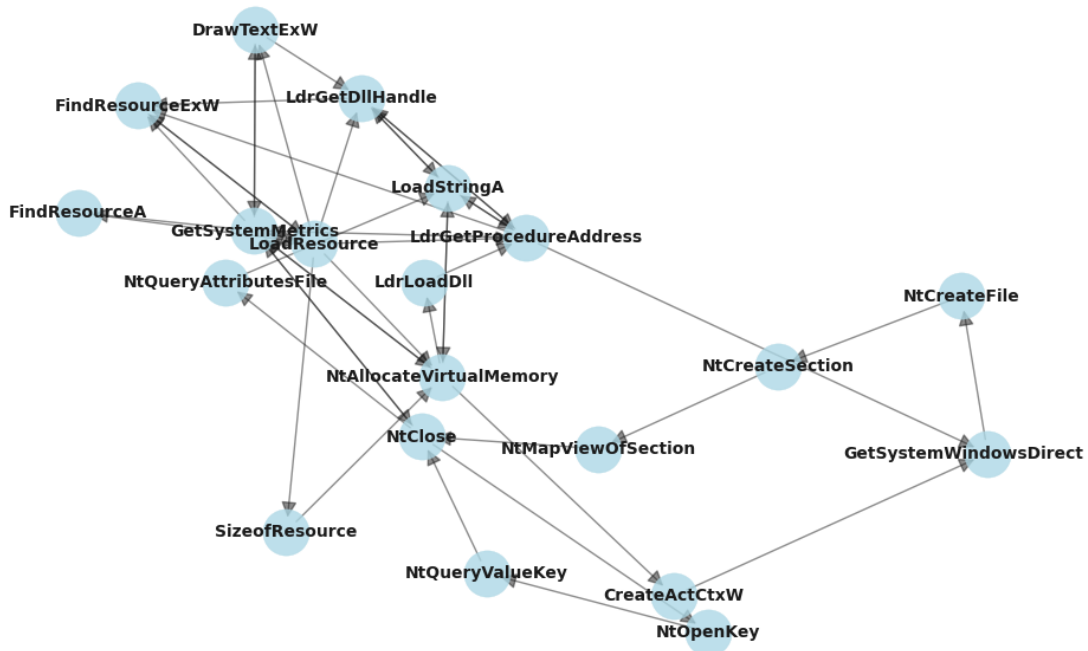
Validation set example:

Malware Execution Graph
Label: 1.0 (1=Malware, 0=Benign)



Test set example:

Malware Execution Graph
Label: 1.0 (1=Malware, 0=Benign)



```
[ ]: data = train_graphs[0]

print(f"1. Node Features Shape: {data.x.shape}")
print(f"    → {data.num_nodes} unique APIs in this sequence")

print(f"\n2. Edge Index Shape: {data.edge_index.shape}")
print(f"    → {data.num_edges} transitions (edges) in the sequence")

print(f"\n3. Max Node Index: {data.edge_index.max().item()}")
print(f"    → Valid range: [0, {data.num_nodes - 1}]")
if data.edge_index.numel() > 0:
    is_valid = data.edge_index.max().item() < data.num_nodes
    print(f"    → {' VALID' if is_valid else ' INVALID - edges point to\nc\n    ↪non-existent nodes'}")
else:
    print(f"    → No edges in this graph")

print(f"\n4. Label Shape: {data.y.shape}")
print(f"    → Label value: {data.y.item()} ({'Malware' if data.y.item() == 1\nc\n    ↪else 'Benign'})")

print(f"\n5. Summary:")
```



```
print(f"    → Graph has {data.num_nodes} nodes and {data.num_edges} edges")
print(f"    → Sparsity: {(data.num_edges / (data.num_nodes * (data.num_nodes - 1))) * 100:.2f}% (if fully connected)")
```

1. Node Features Shape: torch.Size([21, 1])
→ 21 unique APIs in this sequence
2. Edge Index Shape: torch.Size([2, 80])
→ 80 transitions (edges) in the sequence
3. Max Node Index: 20
→ Valid range: [0, 20]
→ VALID
4. Label Shape: torch.Size([1])
→ Label value: 1.0 (Malware)
5. Summary:
→ Graph has 21 nodes and 80 edges
→ Sparsity: 19.05% (if fully connected)

```
[ ]: # Get the raw sequence and the graph
raw_seq_ids = X_train_split[0] # The list of IDs: [117, 118, 119...]
graph = train_graphs[0]

# Check 1: Unique counts
unique_apis_raw = len(set(raw_seq_ids))
# Padding (0) is usually removed in your graph logic, so:
if 0 in raw_seq_ids: unique_apis_raw -= 1

print(f"Unique APIs in raw sequence: {unique_apis_raw}")
print(f"Nodes in Graph: {graph.num_nodes}")
# THESE SHOULD MATCH EXACTLY.

# Check 2: Transitions vs Edges
# Raw edges = length of sequence - 1 (minus padding transitions)
# Note: Your graph combines duplicate edges.
# If raw sequence is A->B->A->B, graph has edges (A,B) and (B,A).
# This check is qualitative: Does graph.num_edges roughly track with sequence
# complexity?
print(f"Raw sequence length: {len([x for x in raw_seq_ids if x != 0])}")
print(f"Graph edges: {graph.num_edges}")
```

```
Unique APIs in raw sequence: 21
Nodes in Graph: 21
Raw sequence length: 81
Graph edges: 80
```

Q: Do you still have to pad your data? If yes, how? No, the graph-based representation eliminates the need for padding entirely.

Unlike FFNNs or RNNs, which require fixed-size dense tensors, Graph Neural Networks (GNNs) operate on sparse structures. In our implementation:

1. **Topological Flexibility:** Each API sequence is transformed into a directed graph where nodes represent unique API calls and edges represent transitions. Longer or more complex sequences simply result in a higher number of edges or a larger subset of nodes, naturally accommodating variable lengths without “dummy” data.
2. **PyG Batching Strategy:** We leverage PyTorch Geometric’s specialized batching, which merges multiple individual graphs into a single large, disjoint graph.

This approach is significantly more memory-efficient than Task 2 and Task 3, as it avoids any redundant computation on padding tokens and preserves the exact structural relationships of the API execution flow.

Q: Do you have to truncate the testing sequences? Justify your answer. TODO: check

No, you absolutely do not truncate the testing sequences.

- **Justification:** The GNN approach, like the RNN, is designed to work with variable-sized inputs. Each sequence is converted into its own graph structure. The size of this graph (number of nodes and edges) is unique to that sequence.
- The model (e.g., GCNConv) and the pooling layer (`global_mean_pool`) are designed to operate on these variable-sized graphs and produce a single, fixed-size vector representation for the *entire graph* regardless of its size.
- Truncating the sequences would mean removing nodes and edges from our graphs, which would be **throwing away crucial structural information**. This would defeat the entire purpose of using a GNN, which is to learn from the complete graph structure of the API calls.

Q: What is the advantage of modeling your problem with a GNN compared to an RNN? What do you lose?

- **Advantage (What you gain):**
 - **Richer Structure:** An RNN only sees the sequence as a linear chain (A -> B -> C). A GNN, as we’ve defined it, captures more complex relationships. For example, in the sequence A -> B -> A -> C, the GNN sees that node A has connections to both B and C, and that B also connects back to A. This creates a small “graphlet” or structural pattern. The GNN can learn that this *pattern* is malicious, whereas the RNN just processes the linear flow.
 - **Order Insensitivity (in this model):** By using `global_mean_pool`, the model becomes permutation-invariant. It cares about the types of connections and patterns that exist (e.g., “this graph contains a lot of A->B edges”) rather than *when* they happened, which is similar to the baseline but on a structural level.
- **Disadvantage (What you lose):**

- **Strict Sequential Order:** The primary thing you lose is the *strict, temporal, step-by-step order* that the RNN explicitly models. The GNN model (as built here with `global_mean_pool`) treats the graph as a single structure. It knows A is connected to B, but it doesn't explicitly know if that connection happened at step 1 or step 50. The RNN's hidden state, by definition, is different at step 50 than at step 1 because it has processed all prior steps.

1.5.2 GNN Training and Helper Functions

We need new training and prediction functions, as the `PyGDataLoader` returns batches differently from the standard `DataLoader`.

```
[ ]: # Create DataLoader for batching
train_loader = PyGDataLoader(train_graphs, batch_size=32, shuffle=True)
val_loader = PyGDataLoader(val_graphs, batch_size=32, shuffle=False)
test_loader = PyGDataLoader(test_graphs, batch_size=32, shuffle=False)

print(f"Train loader: {len(train_loader)}, Validation loader: {len(val_loader)}, Test loader: {len(test_loader)}")
```

Train loader: 409, Validation loader: 103, Test loader: 204

```
[ ]: # =====
# GNN TRAINING FUNCTION (with Macro F1 tracking)
# =====

def train_gnn_common(model, train_loader, val_loader, optimizer, criterion,
    epochs=50, min_delta=0.00001, patience=20):
    """
    Train a GNN model with early stopping and track macro F1 score.
    """
    device = next(model.parameters()).device

    best_val = math.inf
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
    start_time = time.time()

    for epoch in range(1, epochs + 1):
        model.train()
        train_losses = []
        for data in train_loader:
            data = data.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, data.y.unsqueeze(1).float())
            loss.backward()
```

```

        optimizer.step()
        train_losses.append(loss.item())
    train_loss = float(np.mean(train_losses)) if train_losses else 0.0

    # Validation
    model.eval()
    val_losses = []
    all_preds = []
    all_targets = []
    with torch.no_grad():
        for data in val_loader:
            data = data.to(device)
            outputs = model(data)
            loss = criterion(outputs, data.y.unsqueeze(1).float())
            val_losses.append(loss.item())

            # Collect predictions for F1 score
            # FIX: use view(-1) instead of squeeze() to handle
↪single-element batches
            probs = torch.sigmoid(outputs).view(-1)
            preds = (probs > 0.5).long()

            # FIX: convert to list properly to handle both single and
↪multi-element tensors
            all_preds.extend(preds.cpu().tolist())
            all_targets.extend(data.y.cpu().tolist())

    val_loss = float(np.mean(val_losses)) if val_losses else 0.0
    val_f1_macro = f1_score(all_targets, all_preds, average='macro',
↪zero_division=0)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(val_f1_macro)

    if (epoch % 10 == 0) or (epoch == 1) or (epoch == epochs):
        print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} - Val
↪Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

    # Early stopping logic
    if val_loss < best_val - min_delta:
        best_val = val_loss
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
        trigger = 0
    else:
        trigger += 1

```

```

        if trigger >= patience:
            print(f'Early stopping at epoch {epoch} (best val loss_
↪{best_val:.6f})')
            break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(device)

    return model, history, elapsed

```

```
[ ]: # --- GNN Prediction Helper ---
```

```

def predict_loader_gnn(model, loader, threshold=0.5):
    model.eval()

    # Detect device from model parameters
    device = next(model.parameters()).device

    ys_true = []
    ys_pred = []
    with torch.no_grad():
        for data in loader:
            # Move batch to device
            data = data.to(device)

            out = model(data)
            probs = torch.sigmoid(out).cpu().numpy().ravel()
            preds = (probs >= threshold).astype(int)

            # Get true labels from the DataBatch object
            ys_true.extend(data.y.cpu().numpy().ravel().astype(int).tolist())
            ys_pred.extend(preds.tolist())

    return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)

```

```

[ ]: def plot_cm_task4(model, loader, model_name, save_dir=None):
    """
    Specific for GNNs. Uses your existing 'predict_loader_gnn' logic.
    """
    # Use the GNN specific helper
    y_true, y_pred = predict_loader_gnn(model, loader)

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

1.5.3 GNN Variations

Now we define and train the three GNN architectures as requested by the lab PDF. We will use the same hyperparameters (`vocab_size`, `embed_dim`, `hidden_dim`) for a fair comparison. We also re-use the `pos_weight` from Task 3 to handle the imbalanced dataset.

```
[ ]: # Hyperparameters for all GNNs
VOCAB_SIZE_GNN = len(api_to_id)
EMBED_DIM_GNN = 32
HIDDEN_DIM_GNN = 64
OUTPUT_DIM_GNN = 1
EPOCHS_GNN = 100
PATIENCE_GNN = 15
MIN_DELTA_GNN = 0.0001
LR_GNN = 5e-4 # Learning rate

# Re-compute pos_weight for the GNN training subset
pos_weight_gnn = compute_pos_weight(y_train_split).to(device)

criterion_gnn = nn.BCEWithLogitsLoss(pos_weight=pos_weight_gnn)

print(f"GNN Vocab Size: {VOCAB_SIZE_GNN}")
print(f"GNN Positive Weight: {pos_weight_gnn.item():.4f}") # Use .item() for scalar tensor print
```

Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000
GNN Vocab Size: 260
GNN Positive Weight: 0.2000

1. Simple GCN This model uses Graph Convolutional Networks (GCN).

```
[ ]: # --- 1. GCN Model Definition ---

class GCNmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GCNmodel, self).__init__()

        # Embedding: Maps Global API IDs -> Dense Vector
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GCN Layers
        self.conv1 = GCNConv(embed_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.conv3 = GCNConv(hidden_dim, hidden_dim)

        # Pooling
        self.pool = global_mean_pool
```

```

        # Classifier
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        # x comes in as [num_nodes_in_batch, 1]
        x = x.squeeze(1) # [num_nodes_in_batch]

        # 1. Embedding
        x = self.embedding(x) # [num_nodes, embed_dim]

        # 2. GCN
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv3(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        # 3. Global Pooling
        # Averages node vectors belonging to the same graph
        x = self.pool(x, batch)

        # 4. Classification
        out = self.classifier(x)
        return out

```

```

[ ]: # =====
# GCN: Training with Multiple Learning Rates
# =====

gcn_learning_rates = [1e-4, 5e-4, 1e-3]
gcn_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

# Storage for results

```

```

gcn_histories = {}
gcn_models = {}
gcn_elapsed_times = {}

for lr, lr_label in zip(gcn_learning_rates, gcn_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GCN with {lr_label}")
    print('='*60)

    # Set seed for reproducibility
    torch.manual_seed(42)
    np.random.seed(42)

    # Create model
    model_gcn_temp = GCNmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

    optimizer_temp = optim.AdamW(model_gcn_temp.parameters(), lr=lr)

    model_gcn_temp, history_temp, elapsed_temp = train_gnn_common(
        model_gcn_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    gcn_models[lr_label] = model_gcn_temp
    gcn_histories[lr_label] = history_temp
    gcn_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GCN with LR=1e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.1204 - Val Loss: 0.1005 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0745 - Val Loss: 0.0731 - Val F1: 0.6072
Epoch 20/100 - Train Loss: 0.0623 - Val Loss: 0.0666 - Val F1: 0.6562

```


Epoch 30/100 - Train Loss: 0.0579 - Val Loss: 0.0632 - Val F1: 0.6878
 Epoch 40/100 - Train Loss: 0.0539 - Val Loss: 0.0607 - Val F1: 0.6809
 Epoch 50/100 - Train Loss: 0.0499 - Val Loss: 0.0581 - Val F1: 0.7123
 Epoch 60/100 - Train Loss: 0.0475 - Val Loss: 0.0566 - Val F1: 0.7107
 Epoch 70/100 - Train Loss: 0.0447 - Val Loss: 0.0553 - Val F1: 0.7363
 Epoch 80/100 - Train Loss: 0.0417 - Val Loss: 0.0530 - Val F1: 0.7286
 Epoch 90/100 - Train Loss: 0.0406 - Val Loss: 0.0514 - Val F1: 0.7355
 Epoch 100/100 - Train Loss: 0.0385 - Val Loss: 0.0518 - Val F1: 0.7461

LR=1e-4 - Training time: 00:06:13

=====
 Training GCN with LR=5e-4
 =====

Epoch 1/100 - Train Loss: 0.1031 - Val Loss: 0.0874 - Val F1: 0.4905
 Epoch 10/100 - Train Loss: 0.0530 - Val Loss: 0.0579 - Val F1: 0.7034
 Epoch 20/100 - Train Loss: 0.0387 - Val Loss: 0.0512 - Val F1: 0.7417
 Epoch 30/100 - Train Loss: 0.0322 - Val Loss: 0.0513 - Val F1: 0.7986
 Epoch 40/100 - Train Loss: 0.0267 - Val Loss: 0.0469 - Val F1: 0.7664
 Early stopping at epoch 49 (best val loss 0.043830)

LR=5e-4 - Training time: 00:02:59

=====
 Training GCN with LR=1e-3
 =====

Epoch 1/100 - Train Loss: 0.0957 - Val Loss: 0.0754 - Val F1: 0.5865
 Epoch 10/100 - Train Loss: 0.0466 - Val Loss: 0.0548 - Val F1: 0.7357
 Epoch 20/100 - Train Loss: 0.0305 - Val Loss: 0.0541 - Val F1: 0.7964
 Epoch 30/100 - Train Loss: 0.0245 - Val Loss: 0.0563 - Val F1: 0.7849
 Epoch 40/100 - Train Loss: 0.0213 - Val Loss: 0.0557 - Val F1: 0.8266
 Early stopping at epoch 43 (best val loss 0.047240)

LR=1e-3 - Training time: 00:02:38

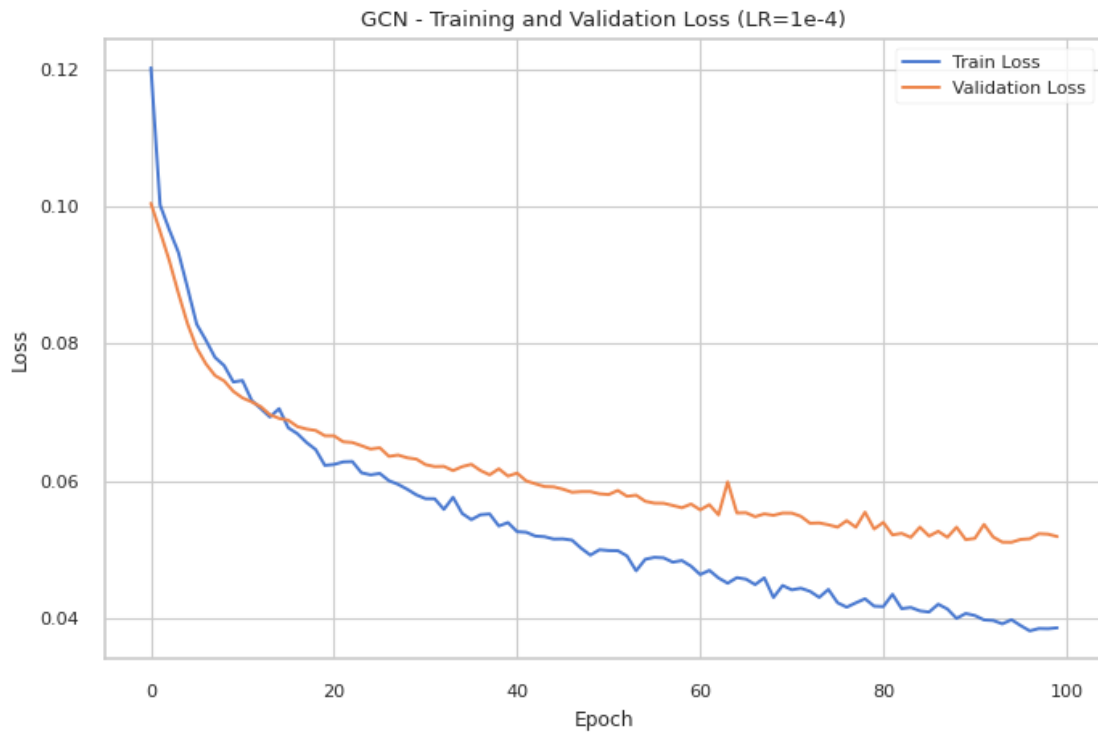
```
[ ]: # =====
# Plot Training/Validation Loss per OGNI learning rate (GCN)
# =====

for lr_label, history in gcn_histories.items():
    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GCN - Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
```

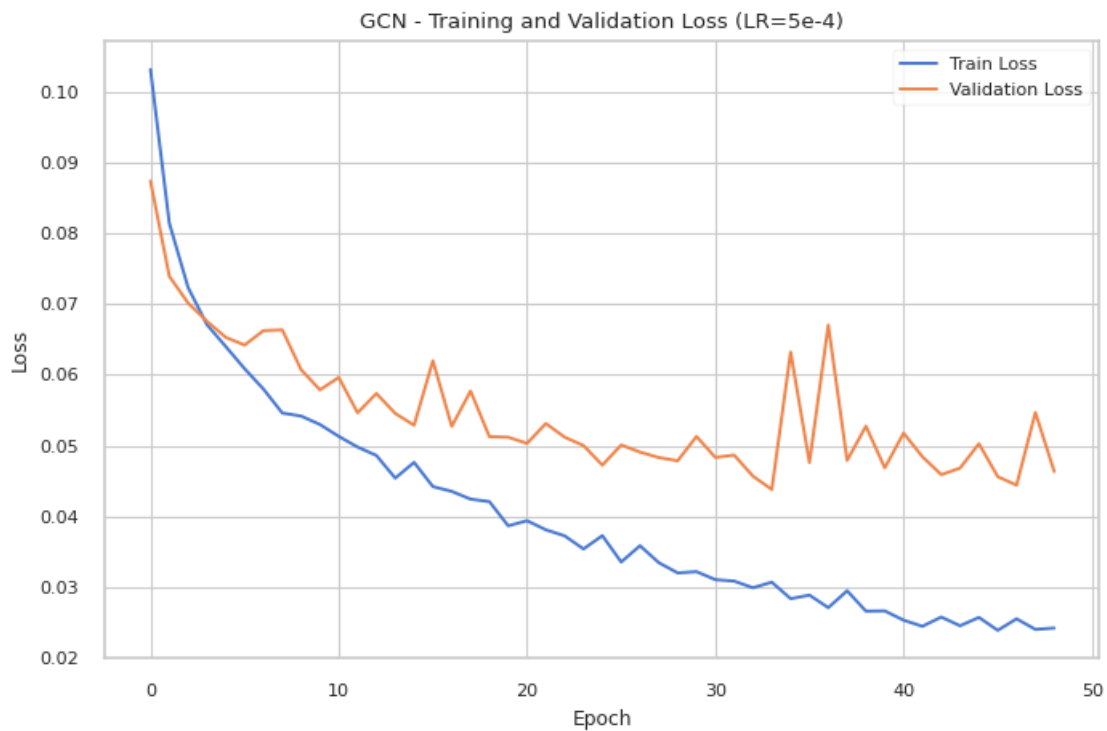
```
plt.grid(True)

# Nome file unico per ogni lr
save_plot(plt.gcf(), f'task4_gcn_loss_curves_{lr_label.replace("=", "")}',
save_dir)
plt.show()
```

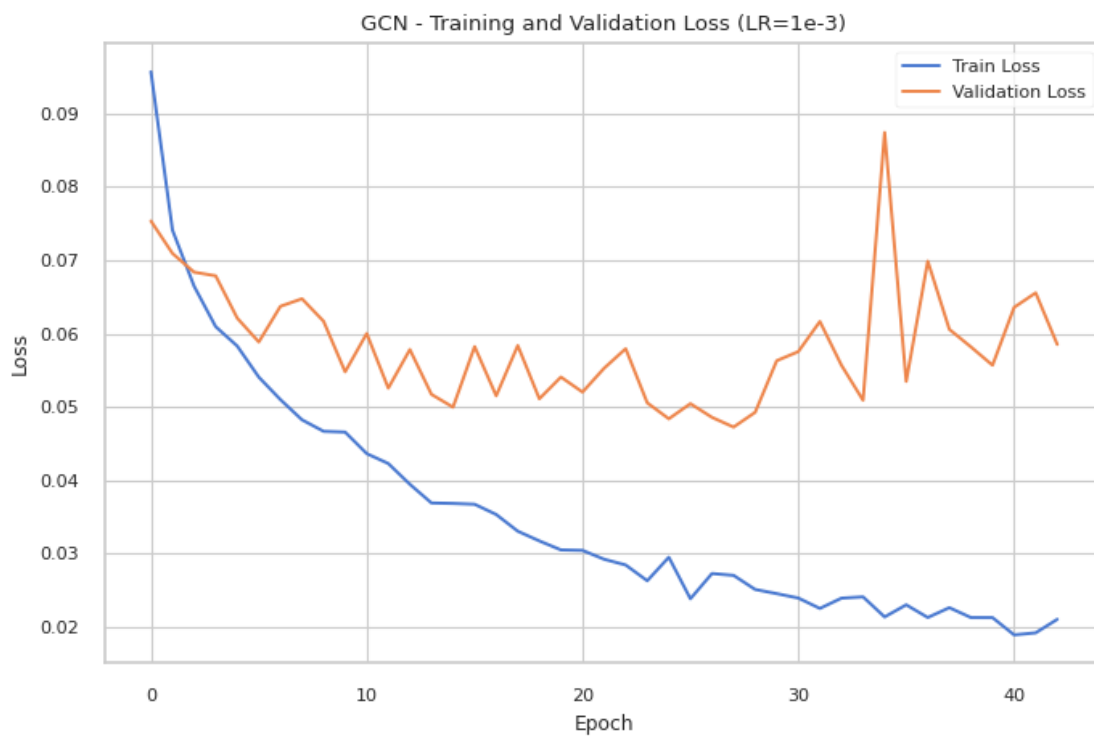
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR5e-4.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-3.png



```
[ ]: # =====
# GCN: Plot Learning Rate Comparison (Validation Loss + Macro F1)
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#1b9e77', '#d95f02', '#7570b3']

# Plot Validation Loss
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,
            color=color)

ax1.set_title('GCN - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

# Plot Validation Macro F1
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
            linewidth=2, color=color)

ax2.set_title('GCN - Validation Macro F1 Score', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gcn_lr_comparison", save_dir)
plt.show()
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_lr_comparison.png



```
[ ]: # --- Evaluate GCN ---
for lr_label, model_gcn in gcn_models.items():
    print(f"\n{'='*60}")
    print(f"Evaluation for GCN - {lr_label}")
    print('='*60)

    y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
    print('\nValidation report (GCN):')
    print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,
↪zero_division=0))

    y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn,
↪test_loader)
    print('Test report (GCN):')
    print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,
↪zero_division=0))
```

```
=====
Evaluation for GCN - LR=1e-4
=====
```

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4009	0.7295	0.5174	122
1	0.9892	0.9577	0.9732	3143
accuracy			0.9492	3265
macro avg	0.6950	0.8436	0.7453	3265
weighted avg	0.9672	0.9492	0.9561	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.3897	0.6543	0.4885	243
1	0.9862	0.9602	0.9731	6262
accuracy			0.9488	6505
macro avg	0.6880	0.8073	0.7308	6505
weighted avg	0.9639	0.9488	0.9550	6505

=====

Evaluation for GCN - LR=5e-4

=====

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4663	0.7951	0.5879	122
1	0.9918	0.9647	0.9781	3143
accuracy			0.9583	3265
macro avg	0.7291	0.8799	0.7830	3265
weighted avg	0.9722	0.9583	0.9635	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.4619	0.7243	0.5641	243
1	0.9891	0.9673	0.9780	6262
accuracy			0.9582	6505
macro avg	0.7255	0.8458	0.7711	6505
weighted avg	0.9694	0.9582	0.9626	6505

=====

Evaluation for GCN - LR=1e-3

=====

Validation report (GCN):

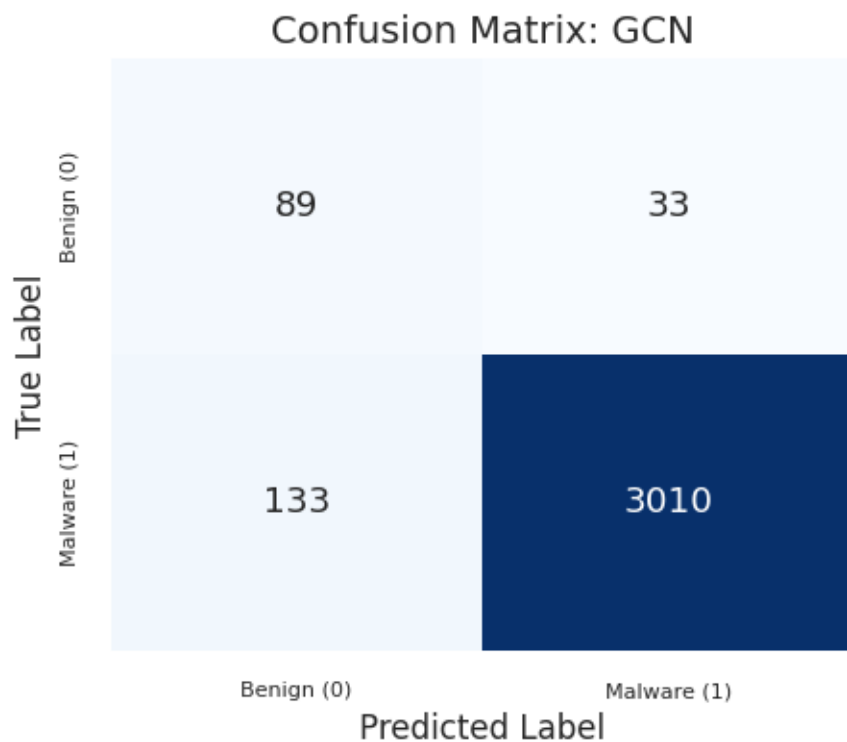
	precision	recall	f1-score	support
0	0.5610	0.7541	0.6434	122
1	0.9903	0.9771	0.9837	3143
accuracy			0.9688	3265
macro avg	0.7757	0.8656	0.8135	3265
weighted avg	0.9743	0.9688	0.9709	3265

Test report (GCN):

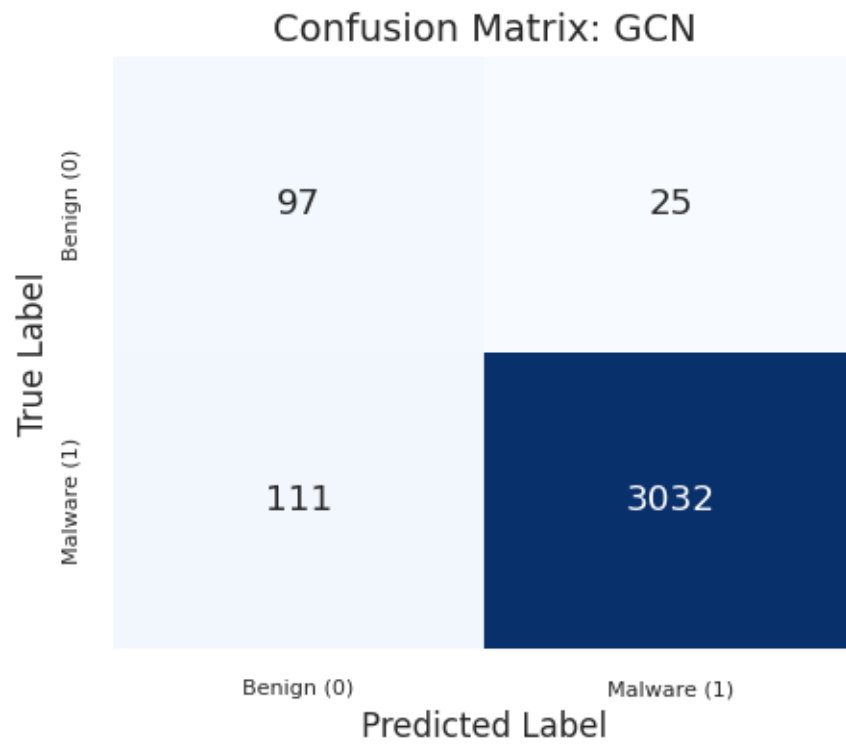
	precision	recall	f1-score	support
0	0.5016	0.6337	0.5600	243
1	0.9856	0.9756	0.9806	6262
accuracy			0.9628	6505
macro avg	0.7436	0.8047	0.7703	6505
weighted avg	0.9676	0.9628	0.9649	6505

```
[ ]: for lr_label, model_gcn in gcn_models.items():  
    plot_cm_task4(model=model_gcn,  
                  loader=val_loader,  
                  model_name="GCN",  
                  save_dir=results_path + 'images/task4_plots/')
```

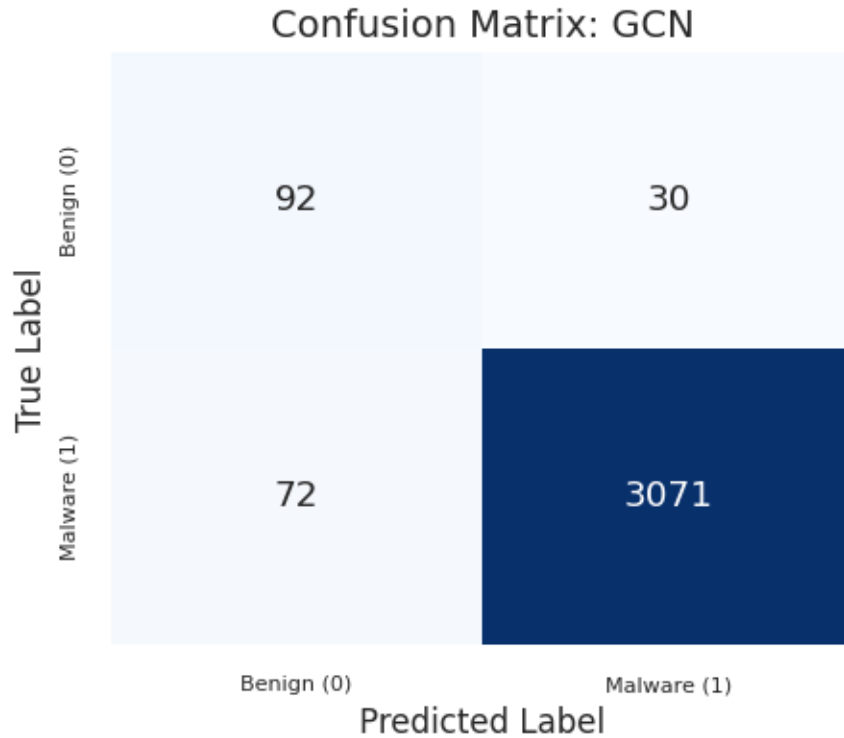
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



```
[ ]: # =====
# GCN: Select Best Model and Evaluate
# =====

# Select best model based on final validation F1;
# change this label if another LR performs better
best_gcn_lr = 'LR=5e-4'
model_gcn = gcn_models[best_gcn_lr]
history_gcn = gcn_histories[best_gcn_lr]
elapsed_gcn = gcn_elapsed_times[best_gcn_lr]

print(f"Selected best GCN model: {best_gcn_lr}")
print(f"Final Val F1 Macro: {history_gcn['val_f1_macro'][-1]:.4f}")

# Evaluate
y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
print('\nValidation report (GCN):')
print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,
    ↪ zero_division=0))

y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn, test_loader)
print('Test report (GCN):')
```

```
print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,
↪zero_division=0))
```

Selected best GCN model: LR=5e-4

Final Val F1 Macro: 0.7688

Validation report (GCN):

	precision	recall	f1-score	support
0	0.4663	0.7951	0.5879	122
1	0.9918	0.9647	0.9781	3143
accuracy			0.9583	3265
macro avg	0.7291	0.8799	0.7830	3265
weighted avg	0.9722	0.9583	0.9635	3265

Test report (GCN):

	precision	recall	f1-score	support
0	0.4619	0.7243	0.5641	243
1	0.9891	0.9673	0.9780	6262
accuracy			0.9582	6505
macro avg	0.7255	0.8458	0.7711	6505
weighted avg	0.9694	0.9582	0.9626	6505

2. GraphSAGE This model uses the GraphSAGE architecture

```
[ ]: # --- 2. GraphSAGE Model Definition ---

class GraphSAGEmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GraphSAGEmodel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # SAGEConv layers
        self.conv1 = SAGEConv(embed_dim, hidden_dim)
        self.conv2 = SAGEConv(hidden_dim, hidden_dim)
        self.conv3 = SAGEConv(hidden_dim, hidden_dim)

        self.pool = global_mean_pool # or global_max_pool

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
```

```

        nn.Dropout(0.5),
        nn.Linear(hidden_dim, output_dim)
    )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv3(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.pool(x, batch)
        out = self.classifier(x)
        return out

```

```

[ ]: # =====
# GraphSAGE: Training with Multiple Learning Rates
# =====

sage_learning_rates = [1e-4, 5e-4, 1e-3]
sage_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

sage_histories = {}
sage_models = {}
sage_elapsed_times = {}

for lr, lr_label in zip(sage_learning_rates, sage_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GraphSAGE with {lr_label}")
    print('='*60)

    torch.manual_seed(42)
    np.random.seed(42)

    model_sage_temp = GraphSAGEmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,

```

```

        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

optimizer_temp = optim.AdamW(model_sage_temp.parameters(), lr=lr)

model_sage_temp, history_temp, elapsed_temp = train_gnn_common(
    model_sage_temp,
    train_loader,
    val_loader,
    optimizer_temp,
    criterion_gnn,
    epochs=EPOCHS_GNN,
    min_delta=MIN_DELTA_GNN,
    patience=PATIENCE_GNN
)

sage_models[lr_label] = model_sage_temp
sage_histories[lr_label] = history_temp
sage_elapsed_times[lr_label] = elapsed_temp

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GraphSAGE with LR=1e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.1102 - Val Loss: 0.0954 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0658 - Val Loss: 0.0667 - Val F1: 0.6460
Epoch 20/100 - Train Loss: 0.0535 - Val Loss: 0.0608 - Val F1: 0.6640
Epoch 30/100 - Train Loss: 0.0473 - Val Loss: 0.0558 - Val F1: 0.7122
Epoch 40/100 - Train Loss: 0.0430 - Val Loss: 0.0543 - Val F1: 0.7383
Epoch 50/100 - Train Loss: 0.0386 - Val Loss: 0.0554 - Val F1: 0.6862
Epoch 60/100 - Train Loss: 0.0355 - Val Loss: 0.0525 - Val F1: 0.7156
Epoch 70/100 - Train Loss: 0.0321 - Val Loss: 0.0509 - Val F1: 0.7383
Epoch 80/100 - Train Loss: 0.0296 - Val Loss: 0.0539 - Val F1: 0.7020
Epoch 90/100 - Train Loss: 0.0267 - Val Loss: 0.0497 - Val F1: 0.7535
Epoch 100/100 - Train Loss: 0.0268 - Val Loss: 0.0515 - Val F1: 0.7885

```

```

LR=1e-4 - Training time: 00:05:20

```

```

=====
Training GraphSAGE with LR=5e-4
=====

```

```

Epoch 1/100 - Train Loss: 0.0974 - Val Loss: 0.0829 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0414 - Val Loss: 0.0528 - Val F1: 0.7157
Epoch 20/100 - Train Loss: 0.0289 - Val Loss: 0.0516 - Val F1: 0.7918

```

Epoch 30/100 - Train Loss: 0.0230 - Val Loss: 0.0540 - Val F1: 0.8318
 Epoch 40/100 - Train Loss: 0.0194 - Val Loss: 0.0595 - Val F1: 0.8332
 Early stopping at epoch 42 (best val loss 0.044724)

LR=5e-4 - Training time: 00:02:15

=====
 Training GraphSAGE with LR=1e-3
 =====

Epoch 1/100 - Train Loss: 0.0913 - Val Loss: 0.0794 - Val F1: 0.5415
 Epoch 10/100 - Train Loss: 0.0333 - Val Loss: 0.0486 - Val F1: 0.7424
 Epoch 20/100 - Train Loss: 0.0237 - Val Loss: 0.0528 - Val F1: 0.8172
 Early stopping at epoch 29 (best val loss 0.044007)

LR=1e-3 - Training time: 00:01:31

```
[ ]: # =====
# GraphSAGE: Plot Learning Rate Comparison
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#377eb8', '#e41a1c', '#4daf4a']

for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,
    ↪color=color)

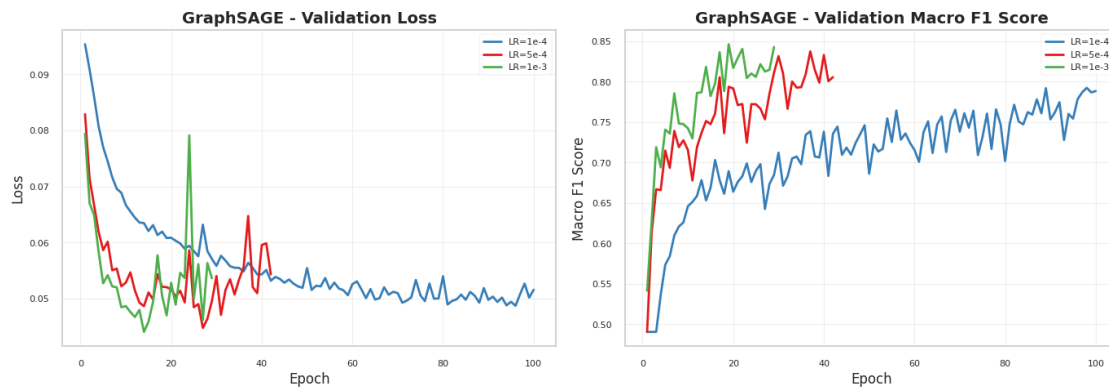
ax1.set_title('GraphSAGE - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
    ↪linewidth=2, color=color)

ax2.set_title('GraphSAGE - Validation Macro F1 Score', fontsize=14,
    ↪fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)
```

```
plt.tight_layout()
save_plot(plt.gcf(), f"task4_sage_lr_comparison.png", save_dir)
plt.show()
```

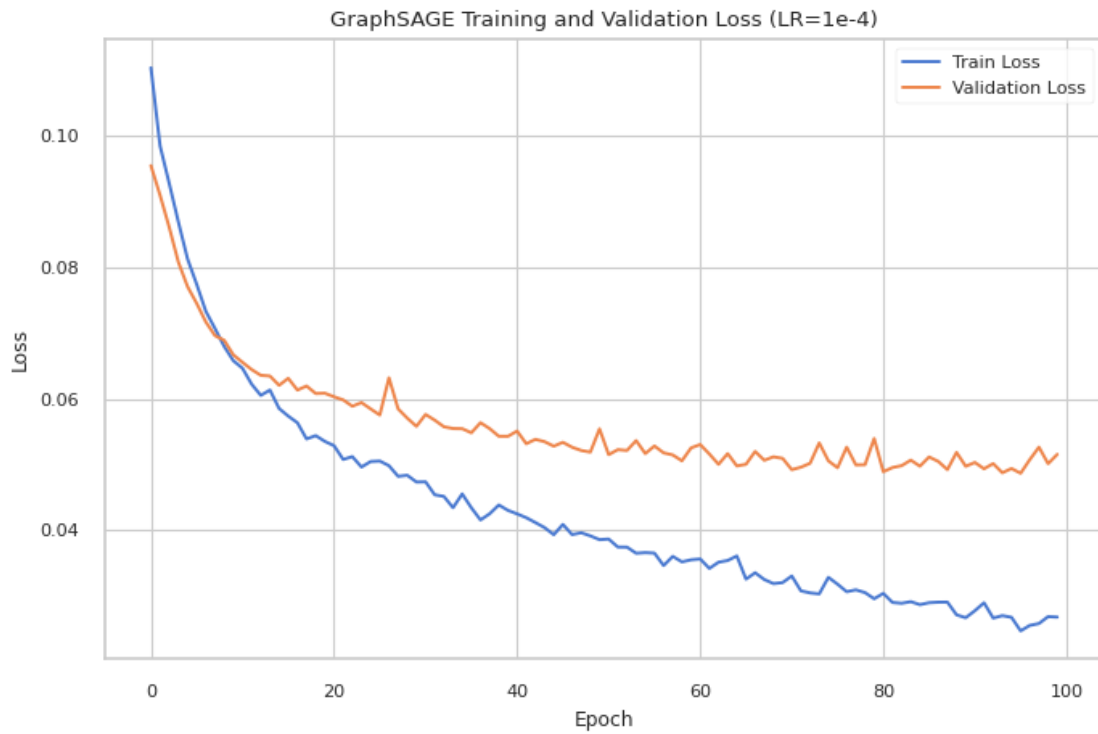
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_sage_lr_comparison.png



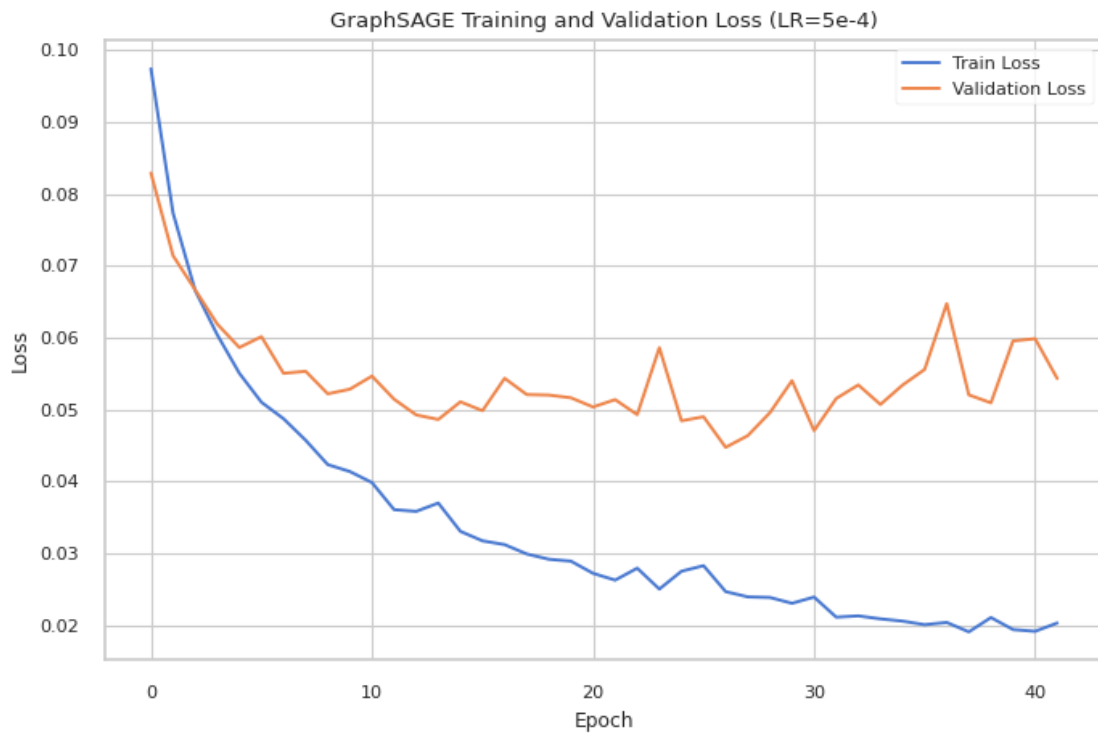
```
[ ]: for lr_label, model_sage in sage_models.items():
    history = sage_histories[lr_label] # <-- Prendi la history per questo LR

    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GraphSAGE Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    save_plot(plt.gcf(), f'graphsage_loss_curves_{lr_label.replace(".", "")}',
    ↪save_dir)
    plt.show()
```

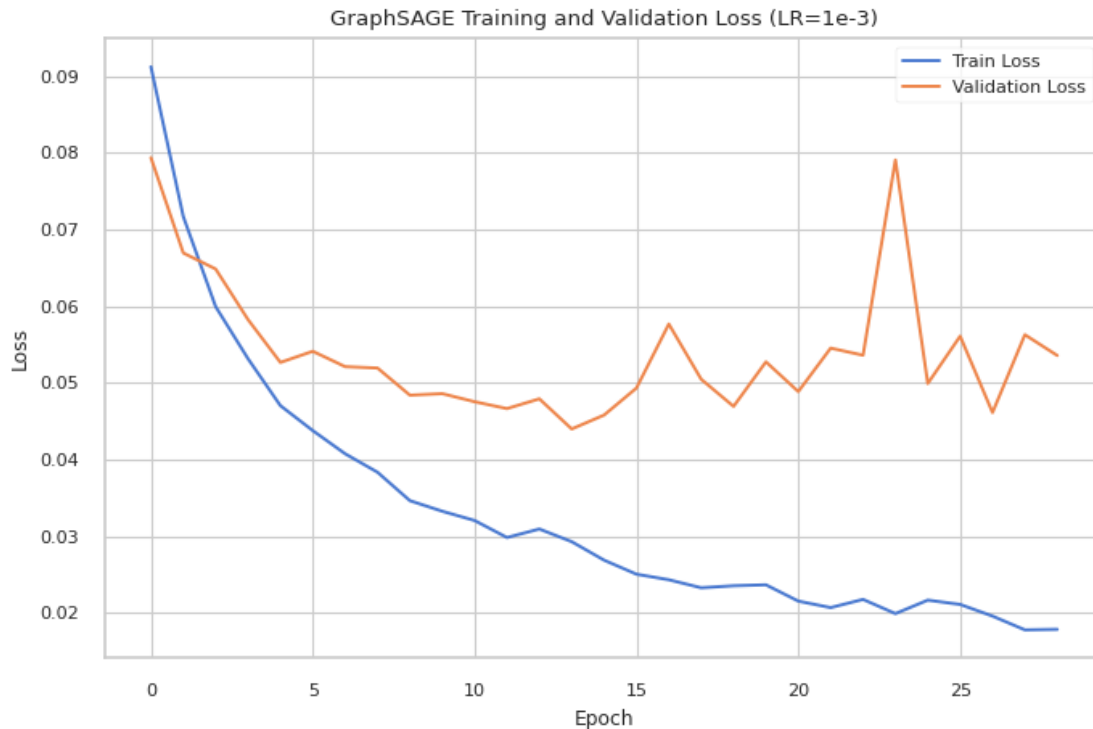
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-4.png



Saved plot: `/content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR5e-4.png`



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-3.png



```
[ ]: # --- Evaluate GraphSAGE ---
for lr_label, model_sage in sage_models.items():
    y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
    print('\nValidation report (GraphSAGE):')
    print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,
    ↪zero_division=0))

    y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage,
    ↪test_loader)
    print('Test report (GraphSAGE):')
    print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,
    ↪zero_division=0))
```

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4125	0.8115	0.5470	122

1	0.9924	0.9551	0.9734	3143
accuracy			0.9498	3265
macro avg	0.7024	0.8833	0.7602	3265
weighted avg	0.9707	0.9498	0.9575	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4321	0.7984	0.5607	243
1	0.9919	0.9593	0.9753	6262
accuracy			0.9533	6505
macro avg	0.7120	0.8788	0.7680	6505
weighted avg	0.9710	0.9533	0.9598	6505

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.3992	0.8115	0.5351	122
1	0.9924	0.9526	0.9721	3143
accuracy			0.9473	3265
macro avg	0.6958	0.8820	0.7536	3265
weighted avg	0.9702	0.9473	0.9558	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4191	0.8107	0.5526	243
1	0.9924	0.9564	0.9741	6262
accuracy			0.9510	6505
macro avg	0.7058	0.8836	0.7633	6505
weighted avg	0.9710	0.9510	0.9583	6505

Validation report (GraphSAGE):

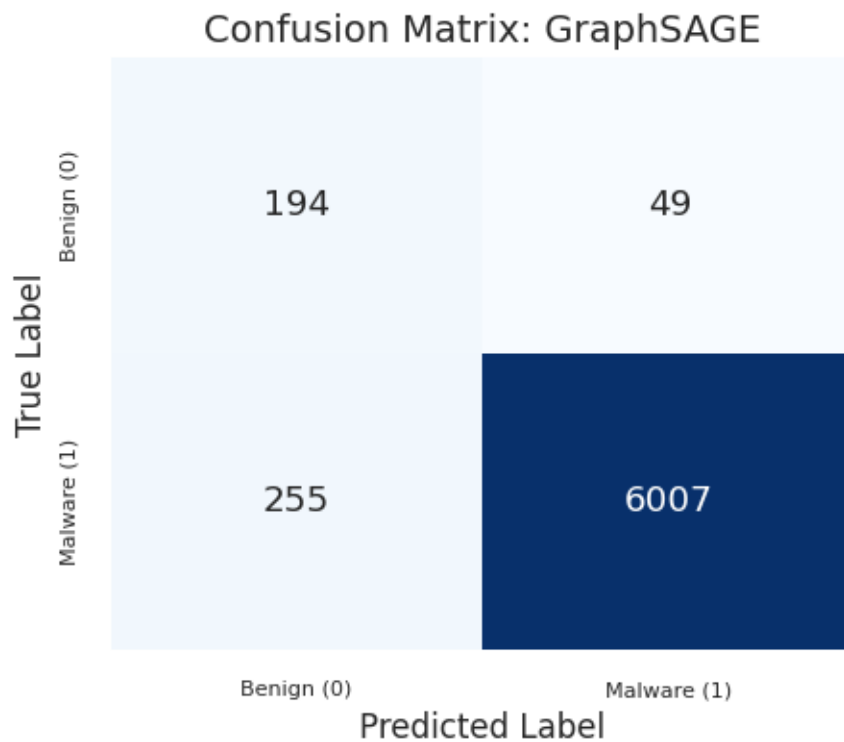
	precision	recall	f1-score	support
0	0.5894	0.7295	0.6520	122
1	0.9894	0.9803	0.9848	3143
accuracy			0.9709	3265
macro avg	0.7894	0.8549	0.8184	3265
weighted avg	0.9745	0.9709	0.9724	3265

Test report (GraphSAGE):

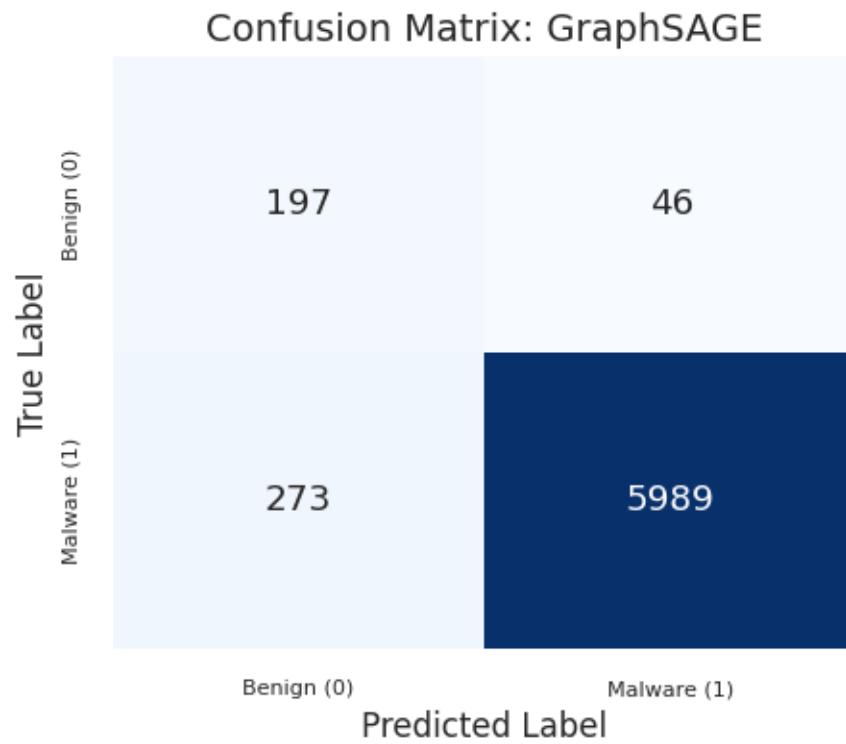
	precision	recall	f1-score	support
0	0.5850	0.7078	0.6406	243
1	0.9886	0.9805	0.9845	6262
accuracy			0.9703	6505
macro avg	0.7868	0.8442	0.8126	6505
weighted avg	0.9735	0.9703	0.9717	6505

```
[ ]: for lr_label, model_sage in sage_models.items():  
    plot_cm_task4(model=model_sage,  
                  loader=test_loader,  
                  model_name="GraphSAGE",  
                  save_dir=results_path + 'images/task4_plots/')
```

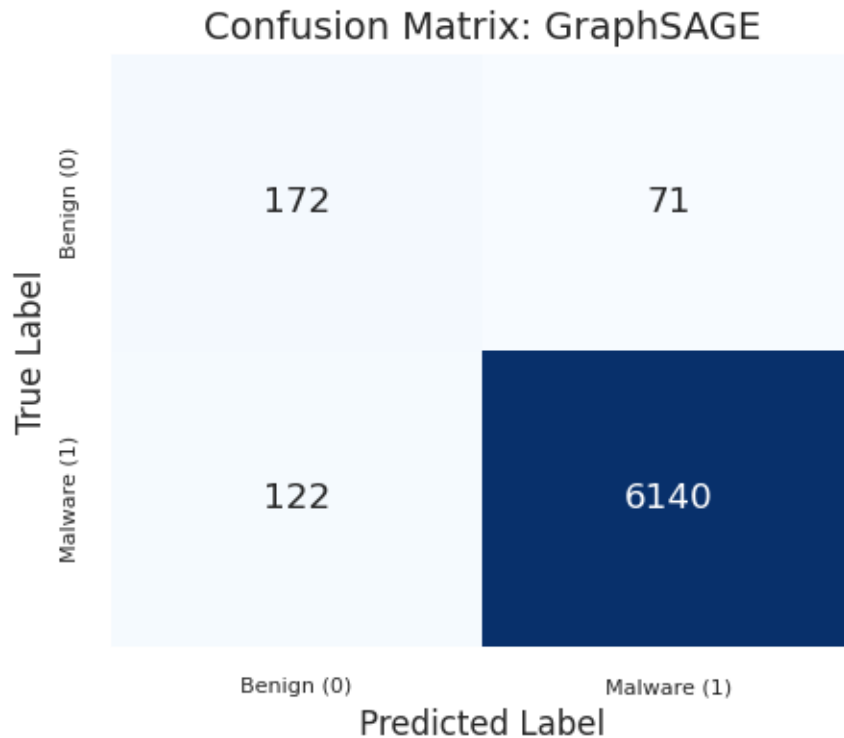
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_graphsage.png



```
[ ]: # =====
# GraphSAGE: Select Best Model and Evaluate
# =====

best_sage_lr = 'LR=5e-4'
model_sage = sage_models[best_sage_lr]
history_sage = sage_histories[best_sage_lr]
elapsed_sage = sage_elapsed_times[best_sage_lr]

print(f"Selected best GraphSAGE model: {best_sage_lr}")
print(f"Final Val F1 Macro: {history_sage['val_f1_macro'][-1]:.4f}")

y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
print('\nValidation report (GraphSAGE):')
print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,
    ↪zero_division=0))

y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage, test_loader)
print('Test report (GraphSAGE):')
print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,
    ↪zero_division=0))
```

Selected best GraphSAGE model: LR=5e-4

Final Val F1 Macro: 0.8057

Validation report (GraphSAGE):

	precision	recall	f1-score	support
0	0.3992	0.8115	0.5351	122
1	0.9924	0.9526	0.9721	3143
accuracy			0.9473	3265
macro avg	0.6958	0.8820	0.7536	3265
weighted avg	0.9702	0.9473	0.9558	3265

Test report (GraphSAGE):

	precision	recall	f1-score	support
0	0.4191	0.8107	0.5526	243
1	0.9924	0.9564	0.9741	6262
accuracy			0.9510	6505
macro avg	0.7058	0.8836	0.7633	6505
weighted avg	0.9710	0.9510	0.9583	6505

3. GAT This model uses the Graph Attention Network (GAT) architecture.

```
[ ]: class GATmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, heads=4):
        super(GATmodel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GAT Layers
        # Layer 1: Output is hidden_dim * heads
        self.conv1 = GATConv(embed_dim, hidden_dim, heads=heads, dropout=0.2)

        # Layer 2: Input is hidden_dim * heads. We set heads=1 for the final
        ↪ graph embedding before pooling.
        # concat=False means we average the heads or just output one head.
        self.conv2 = GATConv(hidden_dim * heads, hidden_dim, heads=1,
        ↪ concat=False, dropout=0.2)

        # Layer 3: Input should now be hidden_dim (output of conv2)
        self.conv3 = GATConv(hidden_dim, hidden_dim, heads=1, concat=False,
        ↪ dropout=0.2)

        self.pool = global_mean_pool
```

```

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        # Conv 1
        x = self.conv1(x, edge_index)
        x = F.elu(x)

        # Conv 2
        x = self.conv2(x, edge_index)

        # Conv 3
        x = self.conv3(x, edge_index)

        # Pooling
        x = self.pool(x, batch)

        # Classifier
        out = self.classifier(x)
        return out

```

```

[ ]: # =====
# GAT: Training with Multiple Learning Rates
# =====

gat_learning_rates = [5e-5, 1e-4, 5e-4]
gat_lr_labels = ['LR=5e-5', 'LR=1e-4', 'LR=5e-4']

gat_histories = {}
gat_models = {}
gat_elapsed_times = {}

for lr, lr_label in zip(gat_learning_rates, gat_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GAT with {lr_label}")
    print(f"{'='*60}")

    torch.manual_seed(42)
    np.random.seed(42)

```

```

model_gat_temp = GATmodel(
    vocab_size=VOCAB_SIZE_GNN,
    embed_dim=EMBED_DIM_GNN,
    hidden_dim=HIDDEN_DIM_GNN,
    output_dim=OUTPUT_DIM_GNN,
    heads=4
).to(device)

optimizer_temp = optim.AdamW(model_gat_temp.parameters(), lr=lr)

model_gat_temp, history_temp, elapsed_temp = train_gnn_common(
    model_gat_temp,
    train_loader,
    val_loader,
    optimizer_temp,
    criterion_gnn,
    epochs=EPOCHS_GNN,
    min_delta=MIN_DELTA_GNN,
    patience=PATIENCE_GNN
)

gat_models[lr_label] = model_gat_temp
gat_histories[lr_label] = history_temp
gat_elapsed_times[lr_label] = elapsed_temp

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
print(f'\n{lr_label} - Training time: {elapsed_str}')

```

```

=====
Training GAT with LR=5e-5
=====

```

```

Epoch 1/100 - Train Loss: 0.1187 - Val Loss: 0.0990 - Val F1: 0.4902
Epoch 10/100 - Train Loss: 0.0725 - Val Loss: 0.0689 - Val F1: 0.6113
Epoch 20/100 - Train Loss: 0.0610 - Val Loss: 0.0597 - Val F1: 0.6945
Epoch 30/100 - Train Loss: 0.0557 - Val Loss: 0.0562 - Val F1: 0.7107
Epoch 40/100 - Train Loss: 0.0509 - Val Loss: 0.0540 - Val F1: 0.7240
Epoch 50/100 - Train Loss: 0.0485 - Val Loss: 0.0532 - Val F1: 0.7259
Epoch 60/100 - Train Loss: 0.0435 - Val Loss: 0.0491 - Val F1: 0.7633
Epoch 70/100 - Train Loss: 0.0432 - Val Loss: 0.0479 - Val F1: 0.7595
Epoch 80/100 - Train Loss: 0.0420 - Val Loss: 0.0484 - Val F1: 0.7560
Epoch 90/100 - Train Loss: 0.0413 - Val Loss: 0.0468 - Val F1: 0.7639
Epoch 100/100 - Train Loss: 0.0392 - Val Loss: 0.0452 - Val F1: 0.7795

```

```

LR=5e-5 - Training time: 00:08:02

```

```

=====
Training GAT with LR=1e-4

```

```

=====
Epoch 1/100 - Train Loss: 0.1088 - Val Loss: 0.0926 - Val F1: 0.5266
Epoch 10/100 - Train Loss: 0.0629 - Val Loss: 0.0612 - Val F1: 0.6938
Epoch 20/100 - Train Loss: 0.0523 - Val Loss: 0.0527 - Val F1: 0.7294
Epoch 30/100 - Train Loss: 0.0465 - Val Loss: 0.0506 - Val F1: 0.7476
Epoch 40/100 - Train Loss: 0.0424 - Val Loss: 0.0464 - Val F1: 0.7644
Epoch 50/100 - Train Loss: 0.0380 - Val Loss: 0.0459 - Val F1: 0.7479
Epoch 60/100 - Train Loss: 0.0343 - Val Loss: 0.0434 - Val F1: 0.7997
Epoch 70/100 - Train Loss: 0.0363 - Val Loss: 0.0442 - Val F1: 0.7556
Epoch 80/100 - Train Loss: 0.0326 - Val Loss: 0.0421 - Val F1: 0.7830
Early stopping at epoch 87 (best val loss 0.041532)

```

LR=1e-4 - Training time: 00:07:00

```

=====
Training GAT with LR=5e-4
=====
Epoch 1/100 - Train Loss: 0.0913 - Val Loss: 0.0726 - Val F1: 0.6441
Epoch 10/100 - Train Loss: 0.0477 - Val Loss: 0.0556 - Val F1: 0.7534
Epoch 20/100 - Train Loss: 0.0387 - Val Loss: 0.0432 - Val F1: 0.7755
Epoch 30/100 - Train Loss: 0.0334 - Val Loss: 0.0464 - Val F1: 0.7629
Epoch 40/100 - Train Loss: 0.0304 - Val Loss: 0.0369 - Val F1: 0.7979
Epoch 50/100 - Train Loss: 0.0284 - Val Loss: 0.0382 - Val F1: 0.7655
Early stopping at epoch 53 (best val loss 0.035931)

```

LR=5e-4 - Training time: 00:04:17

```

[ ]: # =====
# GAT: Plot Learning Rate Comparison
# =====

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#984ea3', '#17becf', '#ff7f00']

for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,
            color=color)

ax1.set_title('GAT - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

for (lr_label, history), color in zip(gat_histories.items(), colors):

```



```

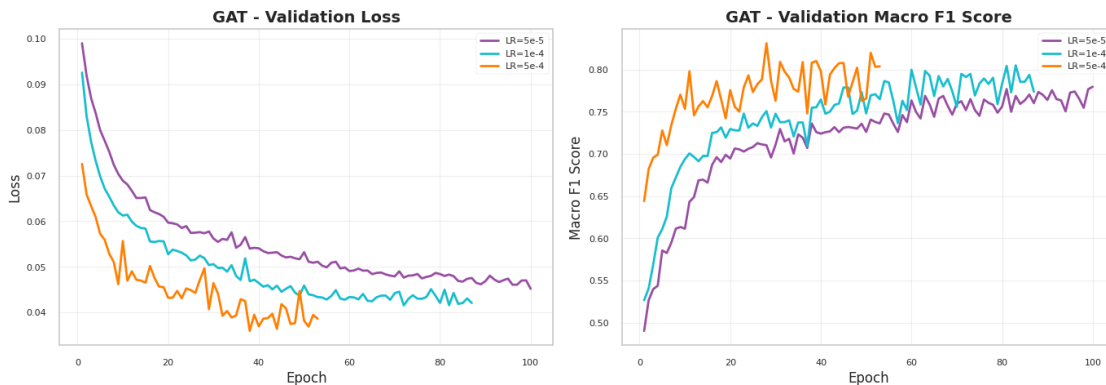
epochs_range = range(1, len(history['val_f1_macro']) + 1)
ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,
        linewidth=2, color=color)

ax2.set_title('GAT - Validation Macro F1 Score', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gat_lr_comparison.png", save_dir)
plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gat_lr_comparison.png.png



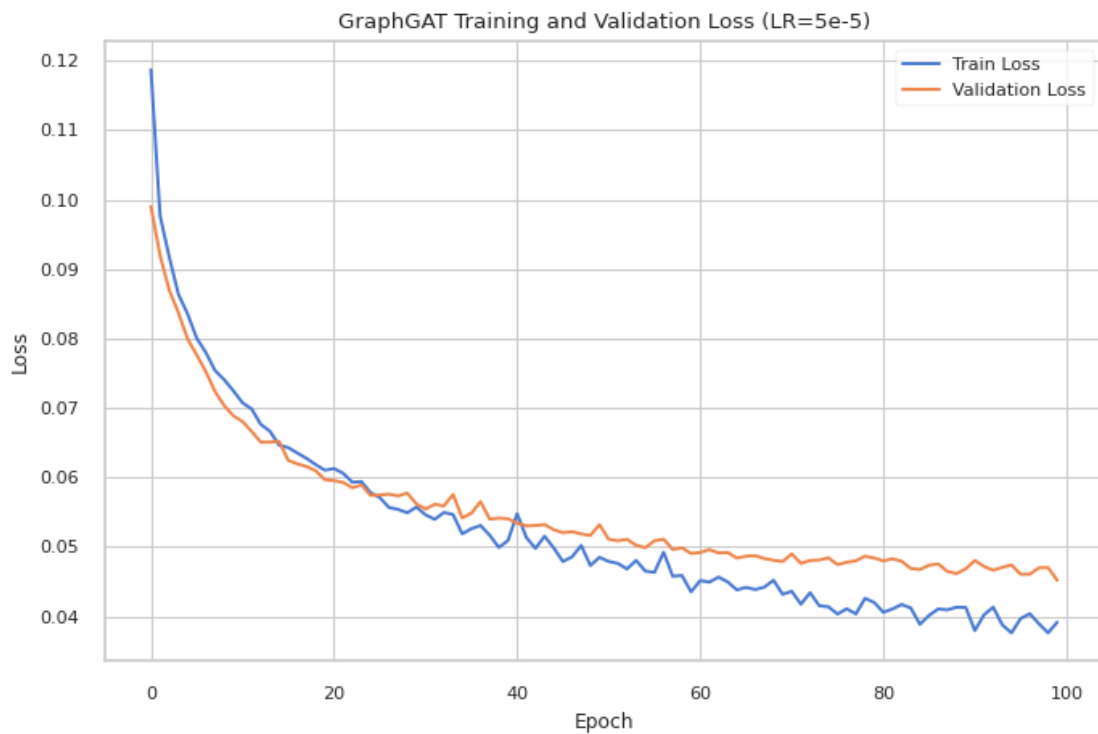
```

[ ]: for lr_label, model_gat in gat_models.items():
    history = gat_histories[lr_label] # <-- Prendi la history per questo LR

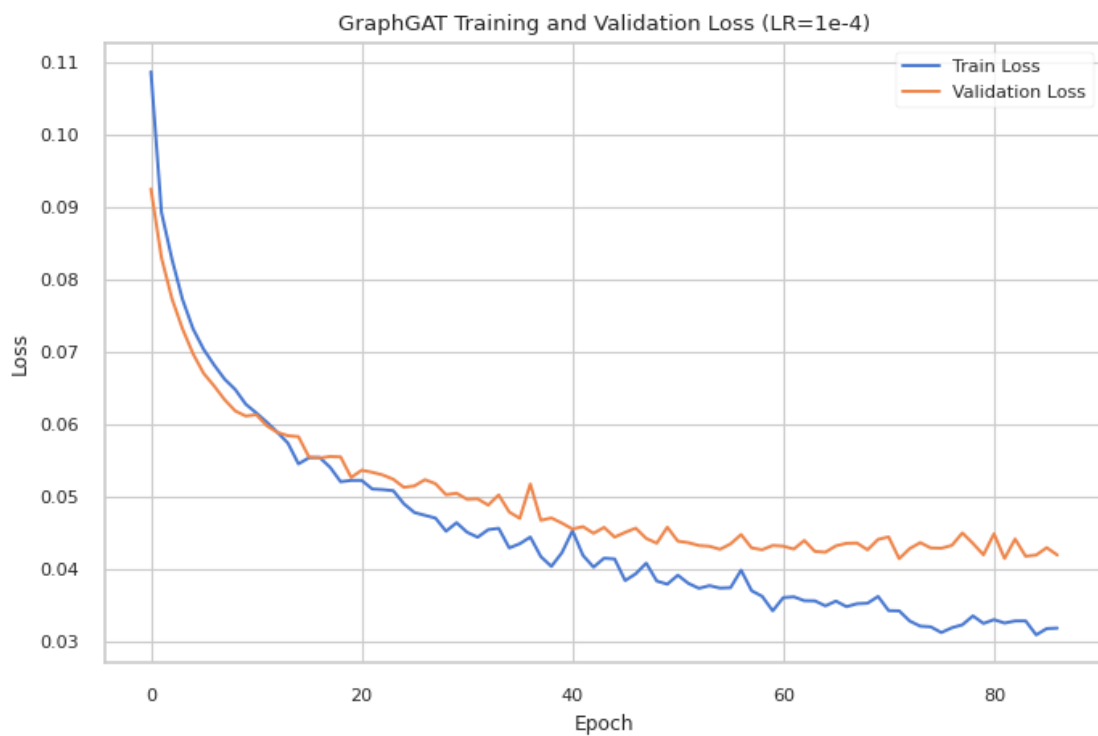
    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GraphGAT Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    save_plot(plt.gcf(), f'graphgat_loss_curves_{lr_label.replace("=", "")}',
            save_dir)
    plt.show()

```

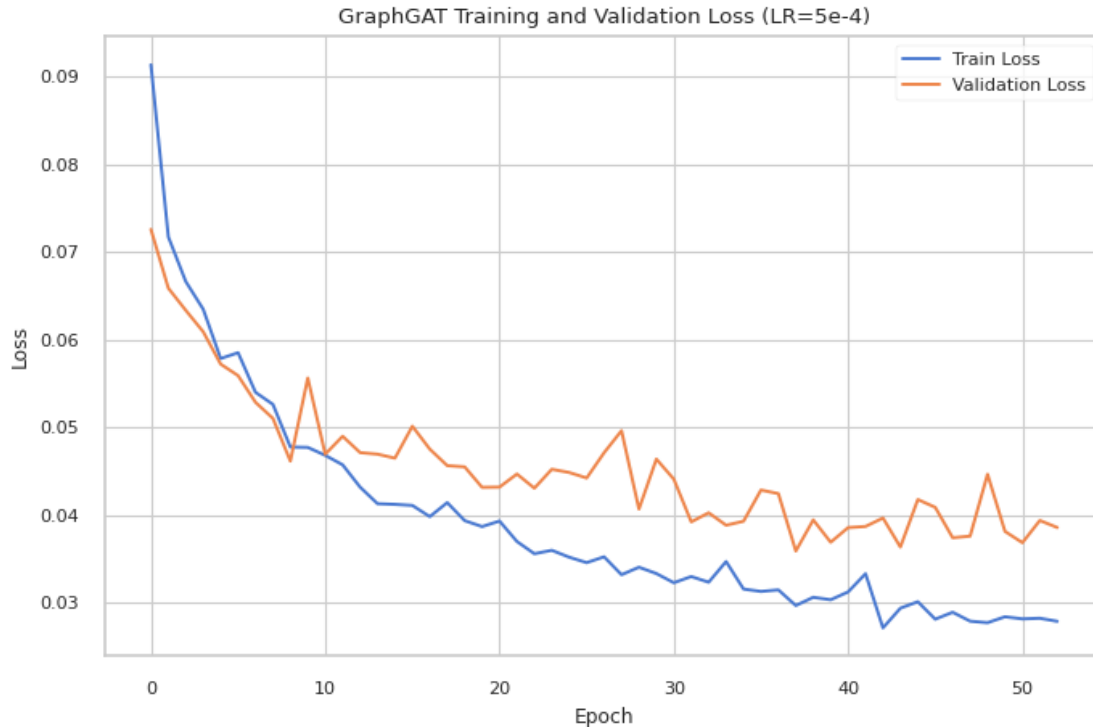
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-5.png



Saved plot: `/content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR1e-4.png`



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-4.png



```
[ ]: # --- Evaluate GAT ---
for lr_label, model_gat in gat_models.items():
    y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
    print('\nValidation report (GAT):')
    print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,
    ↪zero_division=0))

    y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
    print('Test report (GAT):')
    print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,
    ↪zero_division=0))
```

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5030	0.6803	0.5784	122
1	0.9874	0.9739	0.9806	3143

accuracy			0.9629	3265
macro avg	0.7452	0.8271	0.7795	3265
weighted avg	0.9693	0.9629	0.9656	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5127	0.6667	0.5796	243
1	0.9869	0.9754	0.9811	6262

accuracy			0.9639	6505
macro avg	0.7498	0.8210	0.7804	6505
weighted avg	0.9692	0.9639	0.9661	6505

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5316	0.6885	0.6000	122
1	0.9878	0.9765	0.9821	3143

accuracy			0.9657	3265
macro avg	0.7597	0.8325	0.7910	3265
weighted avg	0.9707	0.9657	0.9678	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5535	0.7243	0.6275	243
1	0.9892	0.9773	0.9832	6262

accuracy			0.9679	6505
macro avg	0.7713	0.8508	0.8053	6505
weighted avg	0.9729	0.9679	0.9699	6505

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5304	0.7869	0.6337	122
1	0.9916	0.9730	0.9822	3143

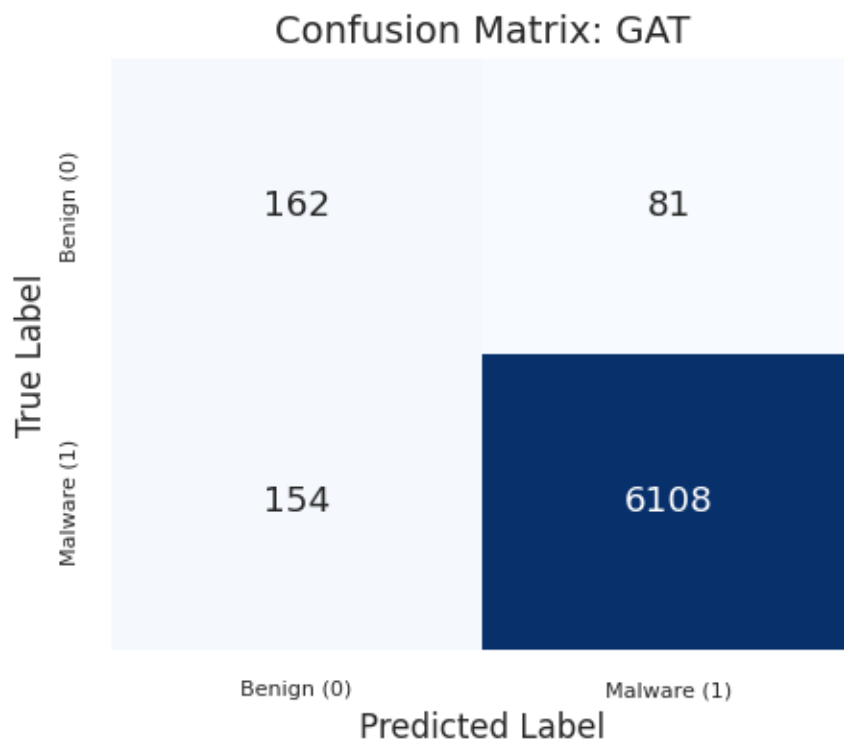
accuracy			0.9660	3265
macro avg	0.7610	0.8799	0.8079	3265
weighted avg	0.9743	0.9660	0.9692	3265

Test report (GAT):

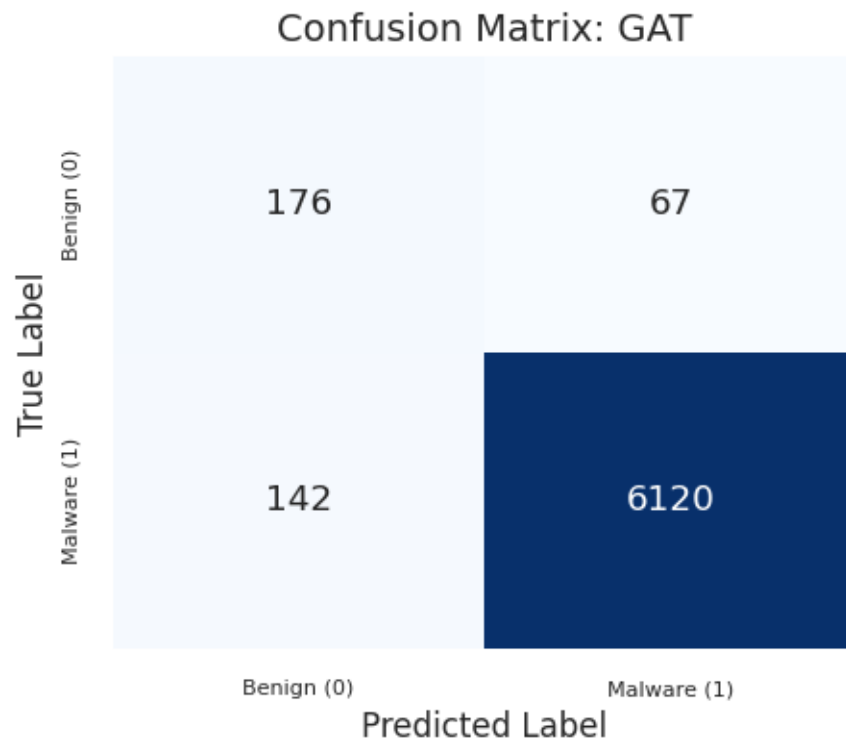
	precision	recall	f1-score	support
0	0.5042	0.7407	0.6000	243
1	0.9898	0.9717	0.9807	6262
accuracy			0.9631	6505
macro avg	0.7470	0.8562	0.7903	6505
weighted avg	0.9716	0.9631	0.9664	6505

```
[ ]: for lr_label, model_gat in gat_models.items():
      plot_cm_task4(model=model_gat,
                    loader=test_loader,
                    model_name="GAT",
                    save_dir=results_path + 'images/task4_plots/')
```

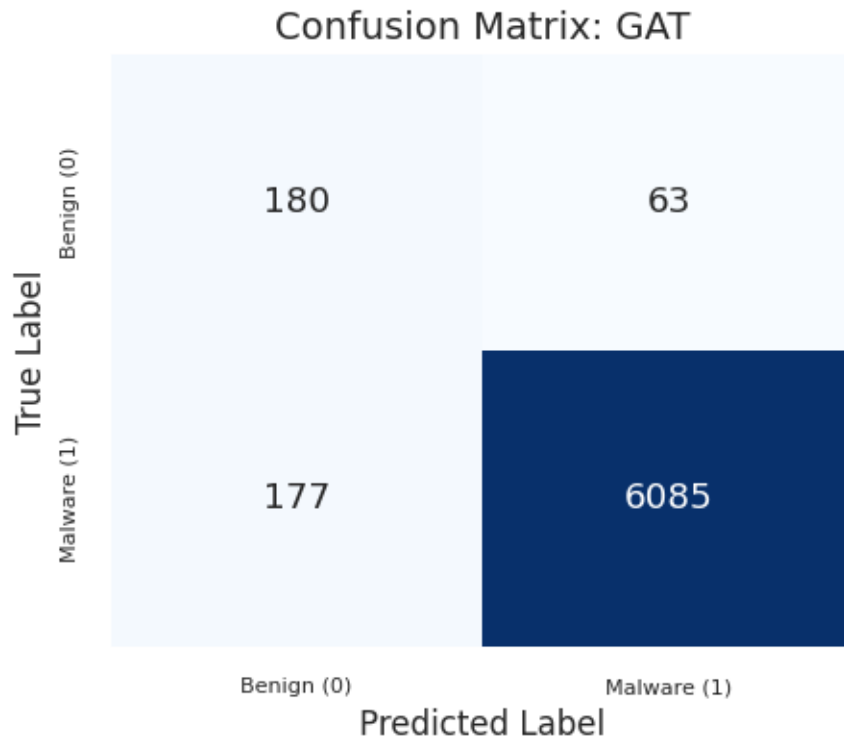
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



```
[ ]: # =====
# GAT: Select Best Model and Evaluate
# =====

best_gat_lr = 'LR=5e-4'
model_gat = gat_models[best_gat_lr]
history_gat = gat_histories[best_gat_lr]
elapsed_gat = gat_elapsed_times[best_gat_lr]

print(f"Selected best GAT model: {best_gat_lr}")
print(f"Final Val F1 Macro: {history_gat['val_f1_macro'][-1]:.4f}")

y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
print('\nValidation report (GAT):')
print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,
    ↳zero_division=0))

y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
print('Test report (GAT):')
print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,
    ↳zero_division=0))
```

Selected best GAT model: LR=5e-4

Final Val F1 Macro: 0.8037

Validation report (GAT):

	precision	recall	f1-score	support
0	0.5304	0.7869	0.6337	122
1	0.9916	0.9730	0.9822	3143
accuracy			0.9660	3265
macro avg	0.7610	0.8799	0.8079	3265
weighted avg	0.9743	0.9660	0.9692	3265

Test report (GAT):

	precision	recall	f1-score	support
0	0.5042	0.7407	0.6000	243
1	0.9898	0.9717	0.9807	6262
accuracy			0.9631	6505
macro avg	0.7470	0.8562	0.7903	6505
weighted avg	0.9716	0.9631	0.9664	6505

```
[ ]: # =====
# GNN Comparison: Validation Macro F1 Score (Best LR for each model)
# =====

fig, ax = plt.subplots(figsize=(10, 6))

# Plot best model for each architecture
best_models_gnn = {
    'GCN': history_gcn,
    'GraphSAGE': history_sage,
    'GAT': history_gat
}

colors_gnn = ['#1b9e77', '#377eb8', '#984ea3']

for (name, history), color in zip(best_models_gnn.items(), colors_gnn):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax.plot(epochs_range, history['val_f1_macro'], label=name, linewidth=2.5,
            color=color)

ax.set_title('GNN Architectures - Validation Macro F1 Score Comparison',
            fontweight='bold',
            fontsize=14)
ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Macro F1 Score', fontsize=12)
```



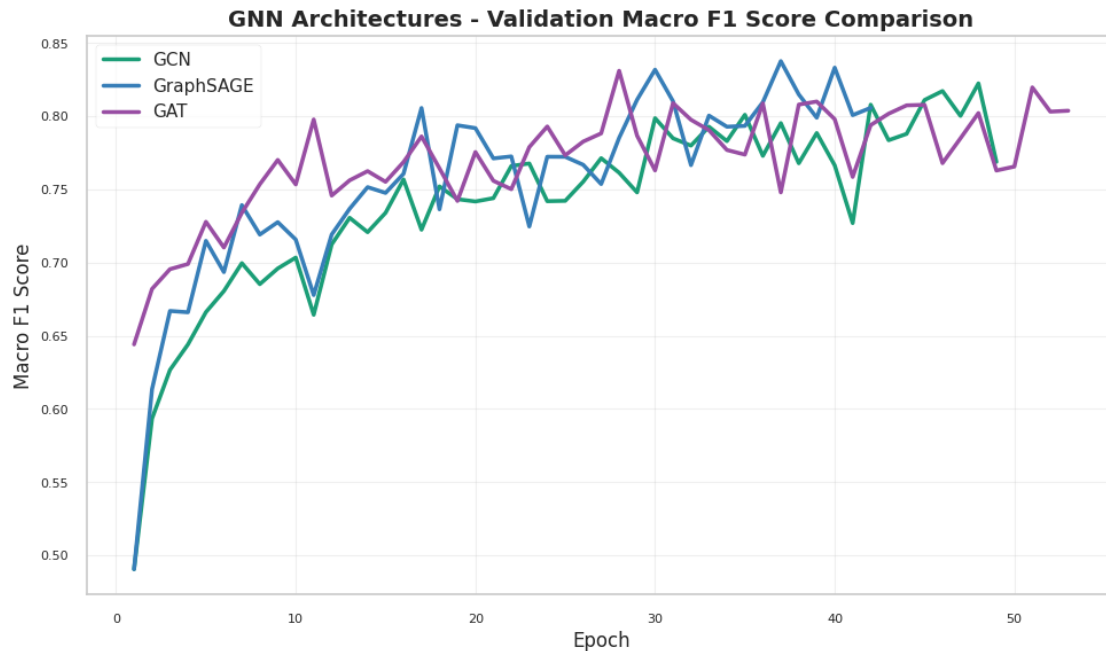
```

ax.legend(fontsize=11)
ax.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gnn_comparison_f1_macro", save_dir)
plt.show()

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gnn_comparison_f1_macro.png



```

[ ]: # =====
# FINAL REPORT: All Architectures Comparison (FFNN, RNN, GNN)
# =====

def get_test_metrics(y_true, y_pred):
    """Extract test metrics from predictions."""
    report = classification_report(y_true, y_pred, output_dict=True,
    ↪ zero_division=0)
    return {
        'precision': report['macro avg']['precision'],
        'recall': report['macro avg']['recall'],
        'f1_score': report['macro avg']['f1-score'],
        'accuracy': report['accuracy']
    }

# Collect metrics for all models

```

```

# Note: Adjust variable names based on your Task 2 and Task 3 results

# --- FFNN (from Task 2) ---
# Assuming you have: y_test_ffnn_true, y_test_ffnn_pred, t_seq (time)
# If variable names differ, adjust accordingly

# --- RNN (from Task 3) ---
# Assuming you have: y_test_true_lstm, y_test_pred_lstm, t_lstm (time)
# If variable names differ, adjust accordingly

# --- GNN (from Task 4) ---
metrics_gcn = get_test_metrics(y_test_true_gcn, y_test_pred_gcn)
metrics_sage = get_test_metrics(y_test_true_sage, y_test_pred_sage)
metrics_gat = get_test_metrics(y_test_true_gat, y_test_pred_gat)

# Build summary table
print("\n" + "="*100)
print("FINAL REPORT: Model Architecture Comparison (Test Set)")
print("="*100)
print(f"{'Architecture':<20} {'Precision':<12} {'Recall':<12} {'F1-Score':<12}␣
    ↳{'Accuracy':<12} {'Time':<15}")
print("-"*100)

# Print GNN results
print(f"{'GCN':<20} {metrics_gcn['precision']:.4f}           {metrics_gcn['recall']:
    ↳.4f}           {metrics_gcn['f1_score']:.4f}           {metrics_gcn['accuracy']:.4f}␣
    ↳           {elapsed_gcn:.2f}s")
print(f"{'GraphSAGE':<20} {metrics_sage['precision']:.4f}           ␣
    ↳{metrics_sage['recall']:.4f}           {metrics_sage['f1_score']:.4f}           ␣
    ↳{metrics_sage['accuracy']:.4f}           {elapsed_sage:.2f}s")
print(f"{'GAT':<20} {metrics_gat['precision']:.4f}           {metrics_gat['recall']:
    ↳.4f}           {metrics_gat['f1_score']:.4f}           {metrics_gat['accuracy']:.4f}␣
    ↳           {elapsed_gat:.2f}s")

print("="*100)

```

```

=====
=====
FINAL REPORT: Model Architecture Comparison (Test Set)
=====
=====

```

Architecture	Precision	Recall	F1-Score	Accuracy	Time
GCN	0.7255	0.8458	0.7711	0.9582	179.62s
GraphSAGE	0.7058	0.8836	0.7633	0.9510	135.01s

GAT	0.7470	0.8562	0.7903	0.9631	257.52s
=====					
=====					

Q: How does each model perform compared to the previous architectures? Can you beat the baseline? Our experimental results confirm that while the frequency-based baseline is competitive in accuracy, deep learning architectures are essential for a balanced and reliable classification.

1. **Beating the Baseline** We successfully outperformed the baseline’s Macro F1-score (0.7223). Although the baseline achieved high accuracy (96.97%), it suffered from a severe bias toward the majority class, failing to identify ~65% of benign samples (Recall: 34.57%). In contrast, our best models—LSTM and GAT—dramatically improved benign detection (Recall > 74%), resulting in significantly higher Macro F1-scores (0.8346 and 0.7903, respectively).
2. **Architecture Comparison** FFNN (Task 2): These models were the weakest (Macro F1: 0.56–0.66), performing below the baseline. Their static nature prevents them from capturing the crucial temporal relationships between API calls, even when using learnable embeddings.

RNNs & LSTM (Task 3): Moving to sequential models provided the largest performance boost. By processing the chronological order of calls, the LSTM achieved the best overall results (Macro F1: 0.8346). Its gating mechanism effectively manages long-term dependencies that simpler RNNs might miss.

GNNs (Task 4): Graph-based models (GCN, GraphSAGE, GAT) proved highly robust. GAT emerged as the top GNN performer (Macro F1: 0.7903). While slightly behind the LSTM in F1-score, GNNs excels at modeling the structural “behavioral signature” of the software, capturing complex non-linear transitions between system calls.
